

2.1 INTRODUCTION

Chapter 1 established the background of our Community Engagement Project and identified a critical community issue: overcrowding at public bus stops serving busy BEST corridors, with special focus on **Dindoshi Bus Depot** (General Arun Kumar Vaidya Marg, Malad East, Mumbai 400097) and **BEST Route 326**. We discussed the need for a low-cost, real-time overcrowding risk signal that combines local passenger sensing with weather and rush-hour context, and presented our proposed solution—an **IoT-Based Bus Stop System for Public Transport Overcrowding Prediction Using Machine Learning**.

This chapter focuses on the extensive analysis and planning required before (and alongside) the build. We begin by reviewing existing research studies on bus crowding prediction, automatic passenger counting (APC), weather–ridership links, and safety-density guidance, then compare current practices to identify research gaps. The chapter then covers requirement-gathering methods, stakeholder analysis, functional and non-functional requirements, a full feasibility study, hardware and software requirements, project schedule, and the development methodology adopted for the prototype.

Taking time for this analysis helps us select effective technologies—such as the **ESP32 DoIT DevKit V1**, **VL53L0X** Time-of-Flight sensor, **DHT22**, **Blynk Cloud**, **Open-Meteo** weather API, Python **brain.py** with a **RandomForestRegressor** (100 trees, `random_state=42`), and **Telegram** alerts—while learning from the limitations of past solutions. Rigorous planning ensures the system is practical for a public-stop demonstration and honest about estimates such as stop capacity **45** and the V7 boarding correction of **−30**.

2.2 LITERATURE REVIEW

A literature review was conducted to deeply understand existing technologies, methods, and research related to bus crowding prediction, automatic passenger counting, weather-sensitive ridership, IoT sensing at transit stops, and congestion-density guidance. Exploring these established studies was a crucial first step to evaluate how previous researchers moved from purely reactive observation toward data-driven prediction and automated counting. Research papers, conference publications, white papers, and local reports were studied to dissect objectives, methodologies, and technical limitations, with focus on comparing sensing architectures and alert mechanisms suitable for a student CEP deployment. By critically analyzing successes and shortcomings, we identified research gaps—particularly the lack of affordable stop-edge IoT risk scoring that fuses passenger crossings with rain, heat, and rush hour, while preserving operator control instead of unsafe automatic count decay.

Paper 1

Title: Data-driven bus crowding prediction models (2020)

Authors: Taqiya Arabghalizi and Alexandros Labrinidis

This research, published in *ACM Transactions on Data Science*, proposes data-driven models for predicting bus crowding from operational and contextual features rather than treating overcrowding as a purely reactive observation. The study demonstrates that crowding can be forecast and used to improve passenger information and operational awareness. However, the approach assumes rich transit-data ecosystems typical of fleet analytics and does not provide a low-cost, single-stop IoT hardware prototype with local LEDs, Blynk gauges, and Telegram escalation for a community bus shelter.

Paper 2

Title: iABACUS: A Wi-Fi-based automatic bus passenger counting system (2020)

Authors: Michele Nitti, Roberto Girau, and Andrea Floris

Published in *Energies*, this paper presents iABACUS, a Wi-Fi-based Automatic Bus Passenger Counting System that estimates passenger presence by observing device Wi-Fi activity. The authors show that IoT connectivity can support passenger counting without requiring every rider to tap a card. However, Wi-Fi probing depends on device carriage and MAC-address behaviour, and raises privacy and calibration concerns for a public shelter CEP; our project therefore uses a short-range **VL53L0X** Time-of-Flight crossing counter for deterministic local sensing.

Paper 3

Title: To travel or not to travel: “Weather” is the question (2018)

Authors: Sui Tao, Jonathan Corcoran, Francisco Rowe, and Mark Hickman

This *Transportation Research Part C* study examines how weather influences travel decisions. The findings support the idea that rain and related conditions change passenger behaviour and can increase clustering under shelters. However, the paper is a travel-behaviour study rather than an embedded stop-edge risk dashboard; our system operationalises the insight as a binary **+15%** rain bonus using **Open-Meteo** WMO codes (61–67, 80–82, 95–99 = Rain; else Clear).

Paper 4

Title: Assessing machine learning algorithms for near-real time bus ridership prediction during extreme weather (2022)

Authors: Francisco Rowe, Mark Mahony, and Sui Tao

This arXiv study (arXiv:2204.09792) evaluates machine-learning approaches for near-real-time bus ridership prediction under extreme weather. It reinforces that weather features improve predictive usefulness for transit demand. However, it targets ridership modelling pipelines rather than a student-built ESP32 node with transparent stop-capacity risk thresholds (0–40 NORMAL, 41–75 WARNING, >75 CRITICAL) and live Blynk visualisation.

Paper 5

Title: An IoT Transport Architecture for Passenger Counting: A Real Implementation (2021)

Authors: Janine Kniess, Julio Cezar Rutke, and William Alberto Cruz Castañeda

Presented at the 2021 IFIP/IEEE International Symposium on Integrated Network Management, this work implements an IoT architecture for passenger counting using embedded computer-vision methods to detect boarding and alighting. It shows the strength of camera-based APC in controlled deployments. However, camera APC introduces cost, lighting, occlusion, and privacy barriers that are poorly suited to a public-stop CEP demonstration; our design deliberately avoids CCTV as the primary sensor.

Paper 6

Title: Automatic passenger counting (APC) and automatic fare collection (AFC) technology (2021)

Authors: Oregon Department of Transportation / Trillium Solutions (white paper)

This Oregon DOT white paper reviews APC and AFC technologies, benefits, components, data standards, and recommendations for transit agencies seeking better passenger data. It clarifies that professional APC deployments can be accurate but require procurement, integration, and standards discipline. However, agency-grade APC/AFC stacks exceed the budget and scope of a Semester-V CEP prototype, motivating our low-cost VL53L0X + ESP32 approach with honest limits on ghost counting.

Paper 7

Title: Guide to Safety at Sports Grounds, 5th edition (2008)

Authors: Department for Culture, Media and Sport (UK)

The Green Guide defines crowd-density bands used in sports-ground safety, treating approximately 3–4 people/m² as congested and about 4/m² as an upper safe reference. Our project

uses this only as an **analogy** for congestion risk—not a claim that a bus shelter is a sports ground. Combined with an estimated shelter area of $\approx 16 \text{ m}^2$ ($\approx 6.0 \text{ m} \times 2.7 \text{ m}$), density $\approx 2.78/\text{m}^2$ yields reasoned capacity **45**. However, 45 remains a **conservative estimate, not a measured on-site survey number**, and MMRDA Bus-Q Shelter material is cited for programme existence only, not for these dimensions.

Paper 8

Title: Five BEST bus terminals to get makeovers (2017)

Authors: The Hindu (news report)

This report discusses BEST terminal upgrades and cites Dindoshi at approximately **3,243 sq m** with on the order of **8,500 passengers/day**, establishing that Dindoshi is a major community transit node. However, depot-wide footfall is not the same as shelter-edge overcrowding; our risk model therefore uses stop capacity 45 at the boarding edge rather than depot totals.

Paper 9

Title: Dindoshi BEST bus depot reported among highest-commuter contexts (2016)

Authors: Times of India (news report)

Contemporary reporting associated Dindoshi with very high daily passenger movement (on the order of ≈ 1.47 lakh in that reporting context), reinforcing community urgency around crowding. However, newspaper aggregates do not replace a transparent local sensor + formula for shelter risk, which is the gap our IoT node addresses.

Paper 10

Title: Dindoshi station profile and passenger-scale figures

Authors: bestpedia.in (station summary) and MMRDA Bus-Q Shelter programme notes

Independent station summaries list Dindoshi at approximately **3,242.9 sq m** and $\approx 8,331$ **passengers/day**, corroborating high utilisation. MMRDA Bus-Q Shelter references confirm that formal shelter programmes exist in the Mumbai region. However, these sources do not supply a verified measured shelter footprint for our exact sensing point; capacity 45 stays a reasoned estimate, and programme citations must not be misused as dimension proofs.

Across the reviewed literature, researchers agree that crowding can be predicted from data, that automatic passenger counting is feasible through Wi-Fi, cameras, or agency APC hardware, and that weather meaningfully affects ridership and waiting behaviour. However, detailed analysis

shows that many solutions target fleet-scale AVL/APC ecosystems, privacy-sensitive cameras, or expensive industrial stacks rather than a localized Mumbai bus-stop CEP. Student IoT demos often omit weather context, transparent risk caps, persistent logging, and a safe operator workflow for bus departure. Consequently, there remains a clear need for an affordable ESP32-based stop-edge system that publishes to Blynk, scores risk with a locked formula (passenger primary weight; rain +15; heat +10 at $\geq 33^{\circ}\text{C}$; rush hour +10), alerts authorities on Telegram when risk > 75%, logs every cycle to **bus_stop_log.csv**, and uses **V7 Bus Left (-30)** with **V8** time-since-crossing evidence—without automatic decay that could wipe a real crowd mid-rush.

2.3 COMPARATIVE ANALYSIS OF EXISTING SYSTEM

Several methods are currently used to judge or measure bus overcrowding. Most existing solutions either rely on informal visual judgement by staff and passengers, use costly camera/APC deployments, or apply fleet analytics that are unavailable at a single community stop. A comparison is presented below to identify strengths and limitations when applied to Dindoshi / Route 326.

Feature	Manual Visual Judgement	Camera / Vision APC	Fleet AVL–APC Analytics	Proposed System (Our Solution)
Continuous live stop-edge tracking	None (human glance only)	Possible but camera-dependent	Vehicle/route level, not shelter-local	Real-time VL53L0X crossings + Blynk
Weather + heat + rush context	Informal only	Usually not fused at stop	Sometimes in research models	Open-Meteo rain + DHT22 heat + clock
Transparent risk % with thresholds	None	Rare / proprietary	Model-dependent, often opaque	Locked formula; max 98%; CRITICAL >75
Authority phone alert	Verbal / radio only	Optional central software	Operations centre tools	Telegram when risk >75 (60s cooldown)
Operator bus-left correction	Mental reset (error-prone)	Entry/exit cameras if dual view	Door APC boarding/alighting	V7 subtract 30 + V8 minutes evidence
Persistent local log for demo/retrain	None	Vendor databases	Agency data warehouses	bus_stop_log.csv each cycle
Cost for student CEP	Zero tech / high labour	High (cameras, privacy)	Very high	Low-cost ESP32 node + free APIs
Public audible alarm	Whistle/shouting	Varies	Not at shelter edge	Buzzer wired but firmware-disabled

The comparison shows that existing approaches either lack continuous sensing or require expensive privacy-sensitive APC infrastructure. Manual judgement is cheap but cannot produce a shared risk percentage. Camera APC can be accurate yet costly and ethically heavier for a public-stop student demo. Fleet analytics rarely expose a shelter-edge gauge for Route 326 at

Dindoshi.

None of the compared options fully combine low-cost ToF counting, Open-Meteo rain context, DHT22 heat, rush-hour timing, Blynk gauges (V1–V8), Telegram CRITICAL alerts, CSV persistence, and a subtract-30 operator button that preserves stragglers. These limitations justify our proposed system.

2.4 RESEARCH GAP IDENTIFICATION

The literature review and comparative analysis reveal several critical limitations in existing overcrowding solutions for community bus stops.

Research Gaps

- **Fleet vs stop-edge focus:** Many studies emphasise bus-load or network crowding models that need rich AVL/APC feeds, overlooking affordable sensing at a single Mumbai shelter edge.
- **Counting without safe correction:** Single-beam or single-ToF counters suffer ghost occupancy if counts reset to zero after a bus leaves; automatic decay can erase a real crowd mid-rush. A manual –30 correction with V8 evidence is under-documented in student IoT projects.
- **Missing contextual bonuses in prototypes:** Rain, heat ($\geq 33^{\circ}\text{C}$), and rush hour (8–9 AM, 6–7 PM) are often ignored in hardware demos even though research links weather to ridership and waiting discomfort.
- **Opaque scores / no persistence:** Some demos show “ML output” without a viva-defensible formula, cap at 98%, or CSV trail for later retraining.

The proposed system addresses these gaps using an ESP32 node (VL53L0X on I2C GPIO21/22; DHT22 on GPIO4; LEDs on GPIO25/26), Blynk Cloud pins V1–V8, Python brain.py with RandomForestRegressor (100 trees, random_state=42), Open-Meteo weather, Telegram alerts for risk > 75%, and bus_stop_log.csv logging—while documenting honest limits (ghost counting; estimated capacity 45; estimated boarding batch 30; synthetic training pending CSV retraining).

2.5 REQUIREMENT GATHERING METHODOLOGY

To design a practical overcrowding-prediction prototype, multiple requirement-gathering methods were used to understand challenges at Dindoshi and the needs of operators and commuters.

Observation Method

Field observation was carried out around **Dindoshi Bus Depot** serving Route 326. Geo-tagged depot photographs were captured for the report. It was evident that boarding-edge crowding is judged visually, with no shared live risk percentage combining passenger pressure, rain, heat, and rush hour for on-duty staff.

Interview Method

Informal discussions with project stakeholders (guide, co-students, and transit-aware community members) highlighted the need for a dashboard that is simple to read, an alert that reaches authority channels without sounding a public panic buzzer, and a bus-left control that does not wipe remaining passengers.

Interview Questions Asked

Do you regularly use or observe BEST services around Dindoshi / Route 326?

How often does the stop feel overcrowded during your usual travel time?

Would a live risk percentage on a dashboard help staff decide when to intervene?

Should alerts go to authority phones (Telegram) instead of a loud public buzzer?

After a bus leaves, is subtracting an estimated boarding batch safer than resetting the count to zero?

Survey Method

A one-page community survey instrument was prepared (blank form; copies for field use).

Integrity note: survey data collection is in progress for this CEP. Preliminary findings from real signed responses will be included in the final black book. Synthetic responses are **not** presented here as real survey data.

Tools Used

Requirement notes were organised digitally (forms/documents) alongside field photographs and the locked Master Project Document controlling pins, formula, and thresholds so analysis cannot drift from the implemented system.

2.6 STAKEHOLDER ANALYSIS

Stakeholder	Role	Need
Commuters (Route 326 / Dindoshi)	Primary beneficiaries	Safer, less chaotic waiting; indirect benefit from earlier staff response when risk is CRITICAL.

Depot / transit staff	On-site / authority operators	Blynk gauges, V8 evidence, V7 –30 control, Telegram alerts when risk >75%.
College CEP coordinators	Academic stakeholders	Demonstrable IoT+ML artefact with honest limitations and community relevance.
Project students (Om Keluskar, Jashith Agre)	System developers	Build, test, document ESP32 + brain.py + Blynk + CSV + Telegram end-to-end.
Project Guide (Prof. George Thekkevilayil)	Academic supervisor	Monitor progress, ensure technical quality and viva-ready integrity.
Co-ordinator (Prof. Vaishali Mishra)	CEP coordination	Ensure NEP-2020 community-engagement compliance and submission standards.

This analysis shows that commuters benefit indirectly, while staff need clear remote/local indicators and a non-panic alert path. Academic stakeholders require honesty about estimates and sensor limits. These needs drive the hybrid hardware–cloud–ML design.

2.7 FUNCTIONAL REQUIREMENT

These requirements define the specific features and capabilities of the proposed overcrowding prediction system.

- FR-1: The ESP32 shall detect a passenger crossing when two consecutive VL53L0X readings are < 400 mm and increment the passenger count.
- FR-2: The ESP32 shall read temperature/humidity from the DHT22 on GPIO4 and publish temperature to Blynk V4.
- FR-3: The ESP32 shall publish passenger count to Blynk V1 and minutes since last crossing to Blynk V8.
- FR-4: brain.py shall fetch Open-Meteo weather, map WMO rain codes to Rain/Clear, and write weather display to Blynk V2.
- FR-5: brain.py shall compute risk using the locked formula and RandomForestRegressor (100 trees, random_state=42), writing the score to Blynk V3.
- FR-6: The system shall classify risk as NORMAL (0–40), WARNING (41–75), or CRITICAL (>75), noting that exactly 75.0 is WARNING.
- FR-7: On CRITICAL, the red LED (GPIO25) shall activate and a Telegram alert shall be sent with a 60-second cooldown; green LED (GPIO26) indicates NORMAL conditions as implemented.

- FR-8: Blynk V7 push button “Bus Left (–30)” shall set `passengerCount = max(0, passengerCount – 30)`.
- FR-9: `brain.py` shall append timestamp, passengers, temp, weather, risk, and status to `bus_stop_log.csv` every cycle.
- FR-10: The buzzer shall remain firmware-disabled despite being physically wired via an NPN transistor; alerts go to Telegram instead.

The above functional requirements define the core operations of the proposed system so that sensing, scoring, alerting, operator correction, and logging remain consistent with the locked Master Project Document.

2.8 NON-FUNCTIONAL REQUIREMENTS

These requirements define quality attributes ensuring the system is reliable, safe for public space, and practical for academic demonstration.

- **NFR-1: Performance** — Sensor publish and brain cycles shall update Blynk gauges frequently enough for a live demo (count climb, risk rise, V7 response visible within normal demo timing).
- **NFR-2: Public-safety UX** — No firmware audible alarm at the stop; CRITICAL escalation is visual (red LED) plus Telegram to authority.
- **NFR-3: Usability** — Blynk layout shall place location label, V7, V8, gauges (V1 max 45; V3 max 98), weather/temp, status colour, and SuperChart in a supervisor-readable arrangement.
- **NFR-4: Integrity / transparency** — Risk caps (passenger component ≤ 75 ; overall ≤ 98) and thresholds shall be viva-defensible without claiming absolute certainty.
- **NFR-5: Availability** — Prototype node and `brain.py` shall support continuous demo operation while powered and networked.
- **NFR-6: Compatibility** — Stack uses ESP32 Arduino libraries, Blynk Cloud, Python 3, scikit-learn, and Open-Meteo HTTPS access.
- **NFR-7: Scalability (future)** — Architecture should allow additional stops later; cloud database remains future work beyond file logging.
- **NFR-8: Maintainability** — Modular wiring on the 3.3 V rail (SDA21/SCL22, DHT GPIO4, LED 25/26 via 220 Ω) supports student maintenance and viva explanation.

- **NFR-9: Cost-effectiveness** — Components and free APIs (Open-Meteo; Blynk free tier for prototype pins) keep deployment within academic budget.

Non-functional requirements ensure the prototype is not only technically capable but also community-safe, affordable, and honest—critical for CEP trust.

2.9 FEASIBILITY STUDY

A feasibility study determines whether the proposed overcrowding-prediction system can be successfully developed and demonstrated for Dindoshi / Route 326 within the academic CEP constraints.

Technical Feasibility

The system is technically feasible because it relies on well-documented hardware and software: ESP32 Wi-Fi, VL53L0X I2C ranging, DHT22, Blynk virtual pins, Open-Meteo JSON weather, scikit-learn RandomForestRegressor, and Telegram Bot API. Development tools (Arduino IDE, Python) are readily available, making integration achievable for TYIT students.

Economic Feasibility

The project is economically feasible using off-the-shelf ESP32-class components rather than industrial APC. Open-Meteo requires no API key; Blynk Cloud free-tier pins V1–V8 suffice for the prototype; CSV logging needs no paid database. This keeps cost within an academic CEP budget.

Operational Feasibility

Operators need minimal training: watch passenger and risk gauges, check V8 before pressing V7 after a bus leaves, and receive Telegram alerts when risk exceeds 75%. Commuters are not asked to install an app. Firmware-disabled buzzer avoids public panic. Therefore operational adoption for a supervised demo is realistic.

Schedule Feasibility

The work fits the Semester V CEP timeline through phases: requirement analysis, hardware assembly, firmware, brain.py + ML formula, Blynk dashboard, Telegram testing, CSV verification, documentation, and demo rehearsal. Parallelism between wiring and Python development shortens the critical path, with buffer for sensor tuning and network debugging.

2.10 SOFTWARE REQUIREMENT

Software/Tech	Version / Note	Purpose
---------------	----------------	---------

Arduino IDE	2.x / ESP32 core	Firmware for ESP32 DoIT DevKit V1
Blynk Cloud	Virtual pins V1–V8	Dashboard, button V7, gauges, SuperChart
Python 3	3.x	brain.py orchestration loop
scikit-learn	RandomForestRegressor	100 trees, random_state=42
Open-Meteo API	Free / no key	Weather; WMO rain code mapping
Telegram Bot API	Bot token chat	CRITICAL alerts (risk >75, 60s cooldown)
CSV logging	bus_stop_log.csv	File-based persistent database
VS Code / editor	Latest	Python editing and demo preparation

2.11 HARDWARE REQUIREMENT

Component	Specs / Pins	Purpose
ESP32 DoIT DevKit V1	3.3 V rail for peripherals	MCU + Wi-Fi node
VL53L0X ToF	I2C SDA GPIO21 / SCL GPIO22	Passenger crossing count (<400 mm ×2)
DHT22	GPIO4	Temperature/humidity; heat if ≥33°C
Red LED	GPIO25 via 220 Ω	CRITICAL indication
Green LED	GPIO26 via 220 Ω	NORMAL indication
Buzzer + NPN	Wired path	Present but firmware-disabled
Misc power/wiring	Breadboard / USB power	Prototype assembly
Laptop	Student development PC	Flashing firmware + running brain.py

2.12 PROJECT SCHEDULE

The project development is divided into several overlapping phases across a **12-week** CEP timeline, including requirement analysis, system design, hardware assembly, ESP32 firmware (including V7/V8), Blynk dashboard layout, brain.py with Open-Meteo + ML risk + Telegram, CSV logging verification, integration testing, and final documentation. Tasks such as firmware development and Python brain work are scheduled concurrently where possible to maximise efficiency. Short buffer periods near the end accommodate Wi-Fi/Blynk debugging, threshold boundary checks (exactly 75.0 = WARNING), and demo rehearsal. The Gantt chart below summarises the planned schedule.

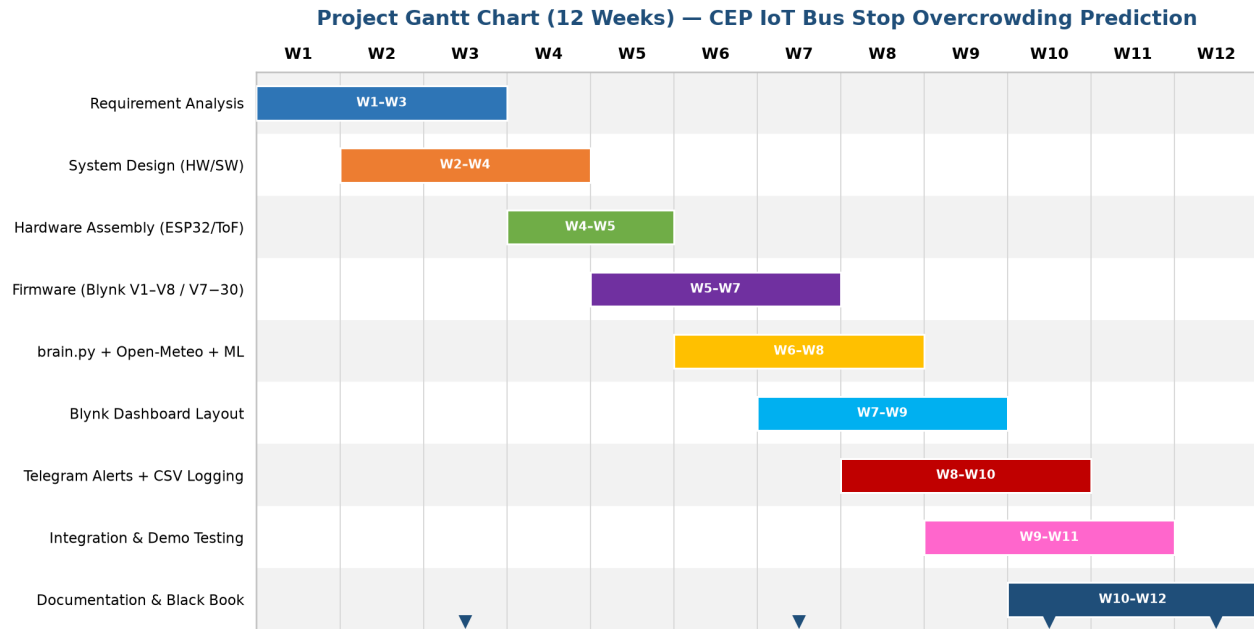


Figure 2.1: Project Gantt Chart (12 Weeks)

Milestones marked on the timeline include requirement freeze (W3), firmware readiness (W7), Telegram + CSV live path (W10), and demo-ready completion (W12). This schedule aligns with internal submission and the implementation demonstration checkpoints (System Ready serial boot, live Blynk, risk >75 with red LED + Telegram, V7 –30, and Excel-visible CSV rows).

2.13 DEVELOPMENT METHODOLOGY

The project follows an **Agile / iterative** development methodology. An IoT+ML CEP requires continuous integration between physical sensing and cloud/software scoring. Instead of building everything once and testing only at the end, we develop and verify in small cycles.

This maps to the schedule: while wiring the ESP32 and validating VL53L0X crossings (<400 mm rule), we simultaneously configure Blynk pins and develop brain.py. Cycles then add rain/heat/rush features, Telegram cooldown, CSV appends, and operator V7/V8 behaviour. Rejected ideas—automatic timer reset and count decay—were tested conceptually and discarded because they can wipe a real crowd mid-rush.

Ultimately, iterative testing ensures hardware, Blynk, ML risk, and alerts communicate correctly for a reliable community demonstration at the Dindoshi / Route 326 context.